

DATA STRUCTURES

LECTURE 01 (INTRODUCTION TO CLASSES)

1

COURSE OBJECTIVE

- To learn efficient storage mechanisms of data for an efficient access
- To Design and Implement various basic and advanced Data Structures.
- To learn various techniques for representation of the data in the memory.
- To develop applications using appropriate data structures
- To improve the logical ability

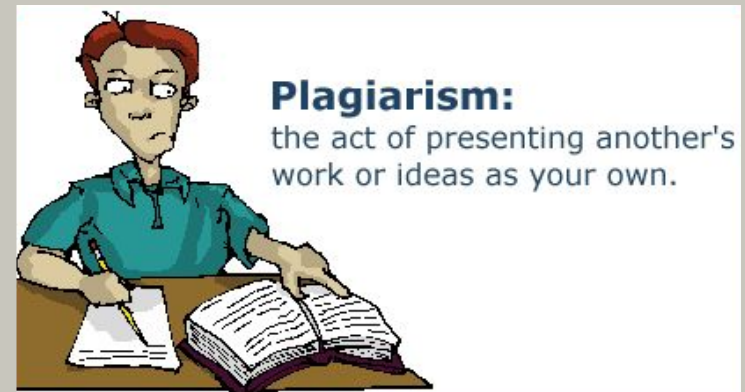
COURSE/ LAB MARKING

- Pre-Lab Tasks (Assignments)
- Lab Tasks (Lab Quiz)
- Post Lab Tasks
- Class Quiz
- Open Lab
- Project
- Sessional I & II
- Final Exam

*Plagiarized stuff will not be appreciated and results in deduction of marks

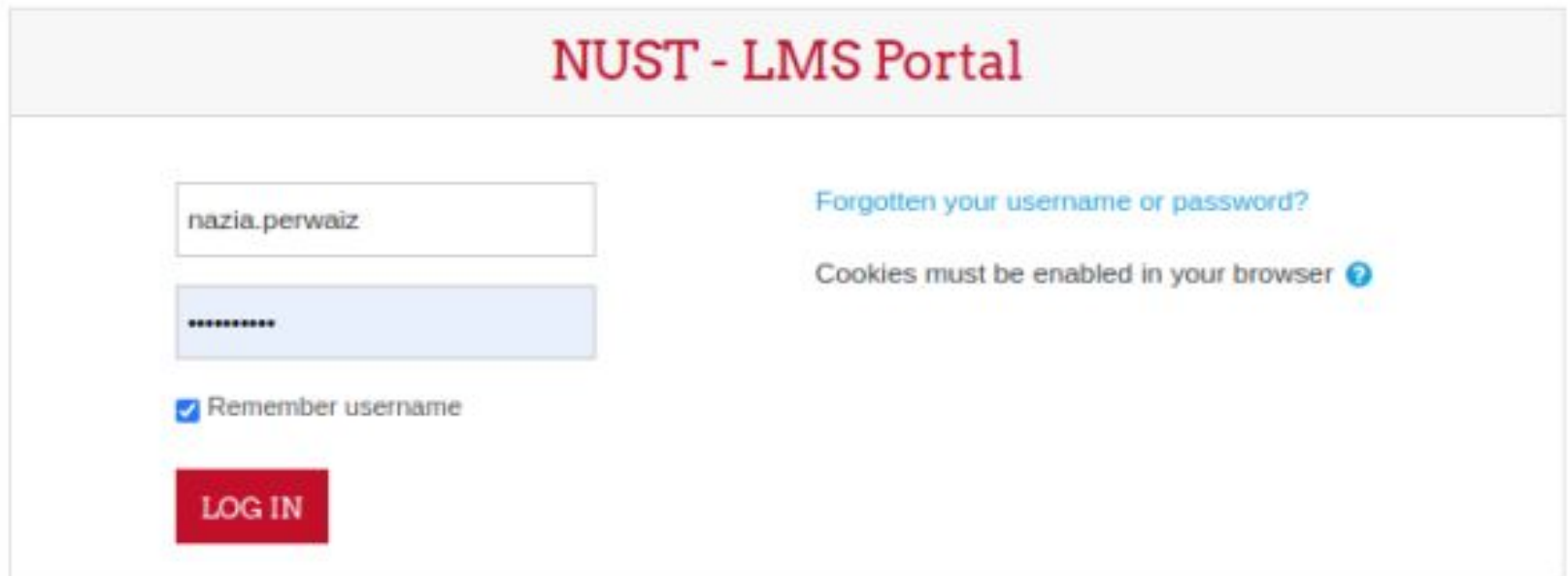
POLICIES

- No extensions will be granted in deadlines.
- Quizzes can be announced/unannounced.
- Exams will be closed book.
- Never cheat.
 - “Better fail NOW or else will fail somewhere LATER in life”
- Plagiarism will also have strict penalties. i.e. will be marked 0
- Reach in time for Class / Lab.



ENROLL YOURSELF

1. Login to LMS portal (<https://lms.nust.edu.pk>)



The screenshot shows the login interface of the NUST - LMS Portal. At the top, a light gray header bar contains the text "NUST - LMS Portal" in a bold, red, serif font. Below this, the main content area is white. On the left side, there are two input fields: the top one contains the username "nazia.perwaiz" and the bottom one contains a masked password "*****". Below the password field is a checkbox labeled "Remember username" which is checked. At the bottom left is a red rectangular button with the white text "LOG IN". On the right side, there is a blue link "Forgotten your username or password?" and a message "Cookies must be enabled in your browser" followed by a blue question mark icon.

ENROLL YOURSELF

The screenshot displays the LMS portal for the National University of Sciences & Technology (NUST). The browser address bar shows the URL `lms.nust.edu.pk/portal/my/`. The top navigation bar is blue and contains the NUST logo, the text "National University of Sciences & Technology", the date "November 10, 2020 09:27:38", and the user name "Nazia Per". Below the navigation bar, the left sidebar contains a list of links: "Dashboard", "Site home" (highlighted with a red box), "Calendar", "Private files", "My courses", "More..." (highlighted with a red box), and "Site administration". The main content area features the "LMS Learning Management System" logo and a "Recently accessed courses" section. This section displays three course cards: "MSEE (Fall 2020) EE800 Stochastic Systems EE-TCN", "MSITE (Fall 2020) ITE803 Pedagogy of Oppressed ITL", and "LMS Training Courses NUST Plagiarism Detection Service".

ENROLL YOURSELF

School of Electrical Engineering and Computer Science (SEECs)

- ▶ SEECs Additional Courses (18)

- ▼ Undergraduate Programs (1)

- ▼ Bachelors of Computer Sciences

- ▶ BS(CS)-2 (Fall 2012 Intake)

- ▶ BS(CS)-1 (Fall 2011 Intake)

- ▶ BS(CS)-3 (Fall 2013 Intake)

- ▶ BS(CS)-4 (Fall 2014 Intake)

- ▶ BS(CS)-5 (Fall 2015 Intake)

- ▶ BS(CS)-6 (Fall 2016 Intake)

ENROLL YOURSELF

Enrolment options

CS100 Fundamentals of ICT BSCS2k20 C

Teacher: **Hania Aslam**

Teacher: **Mehwish Kiran**

Institution: SEECS

Level: UG

Degree: BSCS

Semester: Fall - 2020

Batch: BSCS-2K20

Self enrolment (Student)

No enrolment key required.

ENROL ME

ENROLL YOURSELF

The screenshot shows the LMS interface for the National University of Sciences & Technology (NUST). The browser address bar displays `lms.nust.edu.pk/portal/course/view.php?id=3239`. The page header includes the university name, a date stamp (November 12, 2020 11:36:45), and user information (37 Student SEECS). The main navigation bar features links for NAVIGATION, TRAINING, DOWNLOADS, CONTACT US, and DIGITAL LIBRARY. The left sidebar lists various user options: SandBox_Nazia, More..., Badges, Competencies, Grades, Download center, Dashboard, Site home, Calendar, and Private files. The central content area displays the course title 'SandBox_Nazia' and a breadcrumb trail: Dashboard / Courses / SandBox_Nazia. A red box highlights a button labeled 'Unenrol me from SandBox_Nazia' with a gear icon. Below the course title, there are buttons for 'Activities Summary' and 'Download Center'. On the right, a 'Calendar' widget shows the month of November 2020.

ENROLL YOURSELF

Announcements

Enrollment Code :
u2MH3NAmy

Note: Faculty member needs to share above enrollment code with class students for Self Enrollment in this course on LMS

[Hidden from students](#)

Introduction to the course:

[Insert course introduction/description here]

Learning Objectives:

[Insert learning objectives here]

Course Text / Reference Books:

[Insert text/reference book(s) here]

Description of Evaluation System:

[Insert course evaluation system here]

Lessons Plan:

[Insert detailed lesson plans here]



[Course Outline Document](#)

(PDF document)

[Replace/UPLOAD course outline file and remove this description]

BOOKS

- **Text book**

- Data Abstraction & Problem solving using C++ by Frank M. Carrano, Timothy Henry, 6th Ed.
- Data Structures and Algorithm Analysis in C++ by Mark Allen Weiss, 4th Ed.

- **Reference Books**

- Data Structures & Algorithms in C++ by Michael T. Goodrich, Roberto Tamassia & David Mount, 2nd Ed.

OFFICE HOURS

Office # 01, Faculty Cabin, First Floor, Department of
Computer & Software Engineering,

Day	Timings
Monday	1130 -1300 hrs
Tuesday	1130 – 1300 hrs
Friday	1100 – 1230 hrs

COURSE LEARNING OUTCOMES

Course Learning Outcome (CLOs)		PLOs
CLO1	Analyse C++ programs to determine bugs like dangling pointer, bad pointer, memory leakage and shallow copy etc. without using Compiler.	PLO2
CLO2	Develop C++ programs to learn the use of linear and non-linear data structures and design relevant algorithms for them: (stack, queue, dynamically linked lists, trees, graphs, heap, priority queue, hash tables, sorting algorithms, min-max algorithm).	PLO1
CLO3	Analyse algorithms and data structures for performance comparison in terms of time and space complexity using Asymptotic Analysis.	PLO2
CLO4	Solve open ended complex engineering problems by applying appropriate data structures .	PLO3
CLO5	Develop a project that solves a real problem in an application domain (Networks, virtual hard disk, Data mining, machine learning etc.) by using data structures learnt in this course.	PLO3

**“HARD WORK BEATS
TALENT WHEN TALENT
DOESN'T WORK HARD”**
-TIM NOTKE

```
while(noSuccess)
{
    tryAgain();

    if(Dead)
        break;
}
```


INTRODUCTION TO DATA STRUCTURES

Digital Data



Movies



Music



Photos



Protein
Shapes

DNA

```
gatcttttta ttttaaacgat ctcttttatta gatctctttat taggactcgt atctctctgtg  
gataagtgat tattcaacatg gacgatcata taatttaagga gcatctgtttg ttgtgagtga  
cgggtgatcg tattgcgtat aagctgggat cttaattggca tgttatgcaac agtcaactgg  
cagaatcaag gttgttatgt gcatatctac tggtttttacc ctgcttttaa gcatagttat  
aacatttggc toggcgatc tttgagctaa tttagttaa tttaacctaat ctttgaccac
```



Maps

INTRODUCTION TO DATA STRUCTURES

Data Structures -> Data StructurING

How do we organize information so that we can find, update, add, and delete portions of it efficiently?

DATA STRUCTURE EXAMPLE APPLICATIONS

- How does **Google** quickly find **web pages** that contain a search term?
- How does your **Operating System** track which **memory** (disk or RAM) is free?
- How does the **processor** handles multitasking

WHAT IS A DATA STRUCTURE ANYWAY?

- It's an agreement about:
 - **how to store** a collection of objects in memory,
 - **what operations** we can perform on that data,
 - **the algorithms** for those operations, and
 - how **time and space** efficient those algorithms are.

HAVE YOU EVER USED A DATA STRUCTURE TO STORE YOUR DATA?

- Arrays in C/C++
 - Stores objects sequentially in memory
 - Can access, change, insert or delete objects
 - Algorithms for insert & delete will shift items as needed

C++ VS. C#

C++	C#
Partially object oriented	Completely Object oriented
Main is a global function	Main is a member function of class
Class members having same Access specifiers requires one time declaration	Access specifier must be mentioned before every class member
Destructors are required to deallocate memory	There is less need for destructors, since dynamically allocated memory will get removed automatically
Platform independent	Targeted towards windows OS
Requires a semicolon after the closing brace at the end of a class definition	Do not requires a semicolon after the closing brace at the end of a class definition

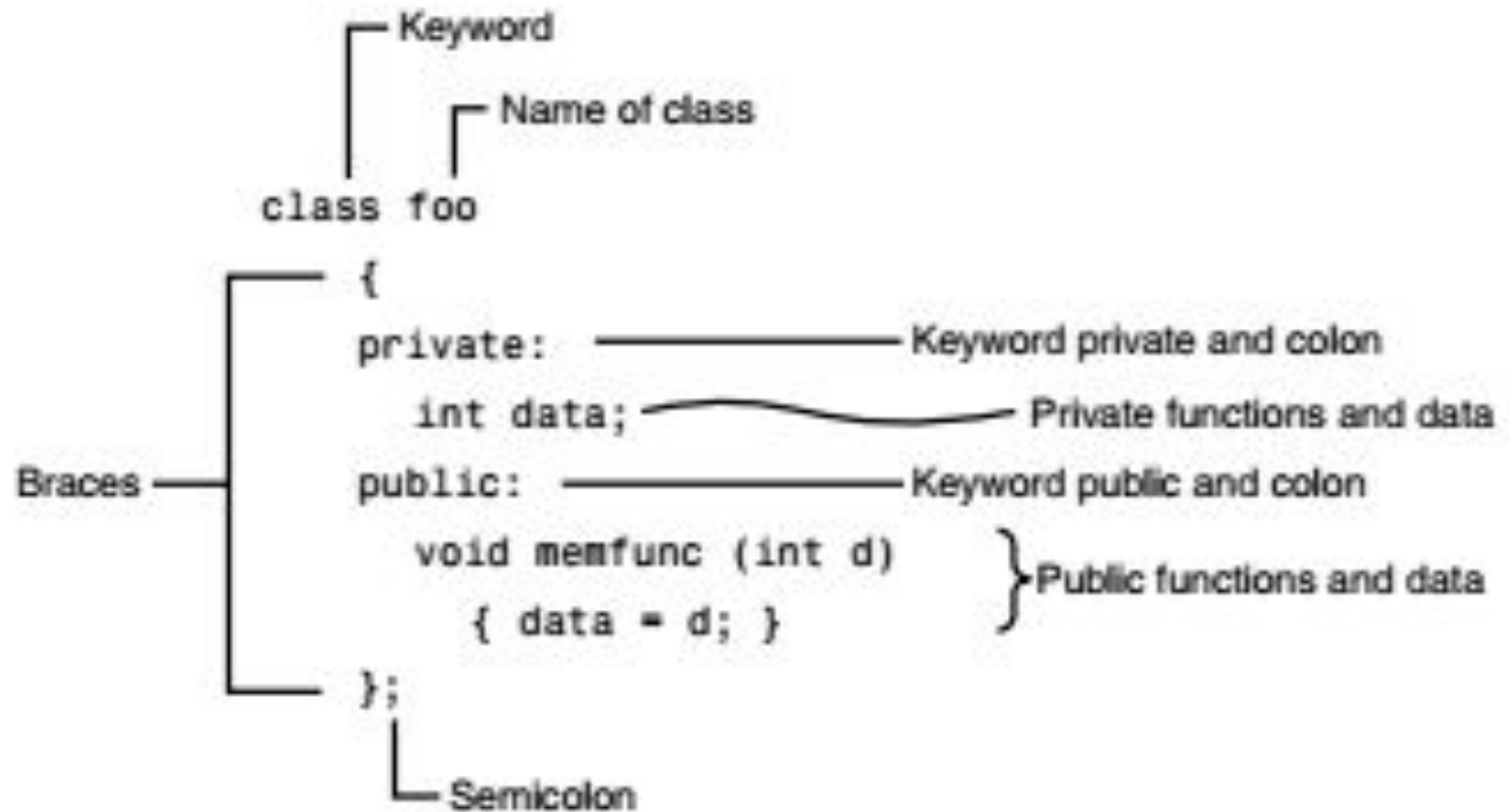
CLASS (ABSTRACT DATA TYPE)

- To define a data type which is a composite of more than one compiler defined data types
- A class serves only as a plan, or a template, or sketch- of a number of similar things (i.e. objects)
- A class is a mechanism for creating **user-defined data types.**

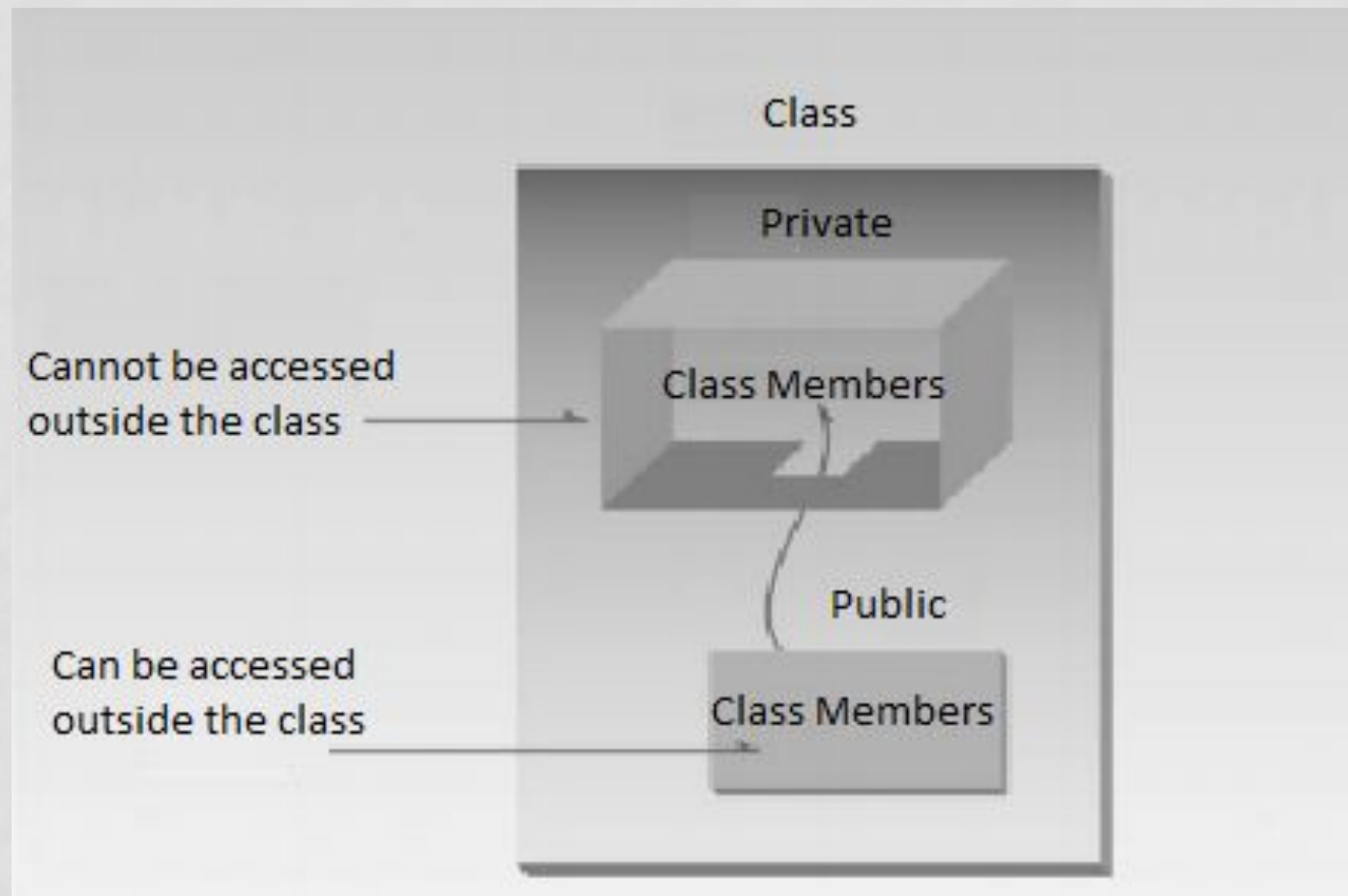
DEFINING A CLASS IN C++

```
class class_name  
{  
private:  
    // private class members  
  
public:  
    // public class members  
  
};
```

STRUCTURE OF A CLASS



WHY WE NEED ACCESS SPECIFIERS



OBJECTS

- A class and an object of that class has the same relationship as a data type and a variable
- All objects with the same characteristics constitute one class.

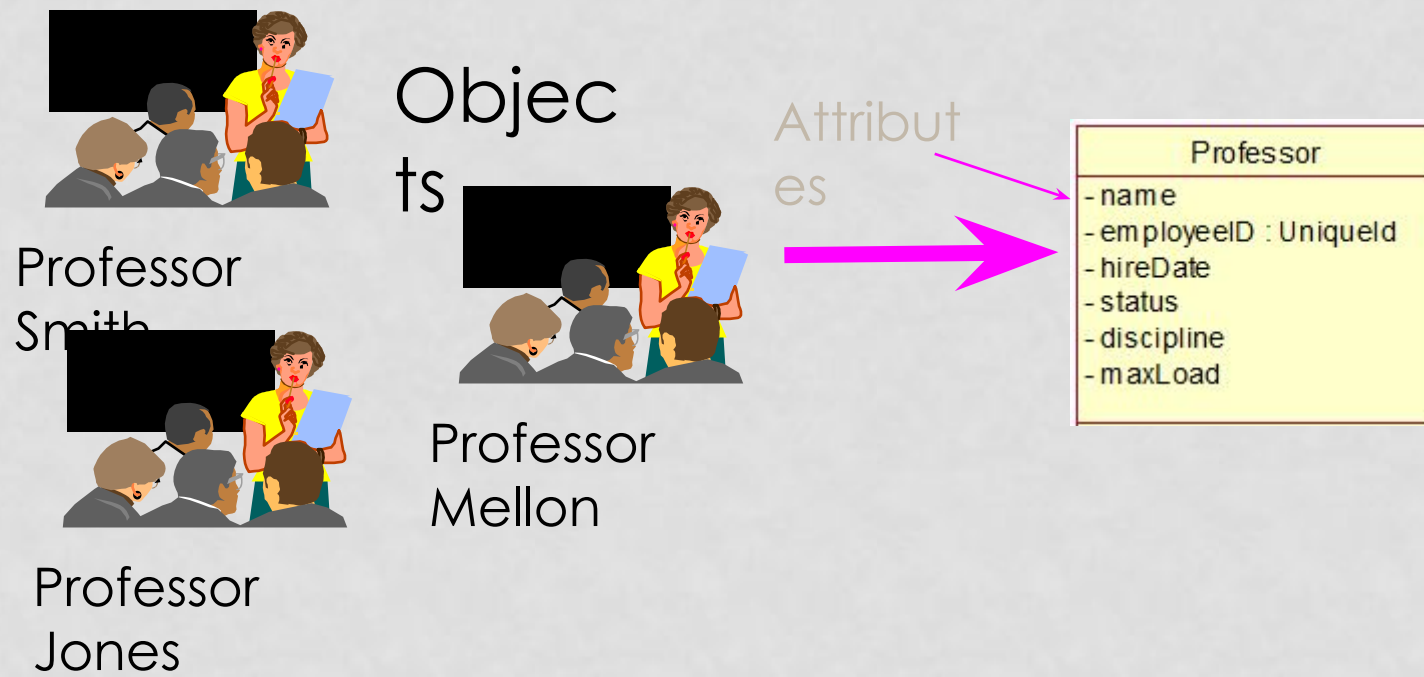
OBJECT INSTANTIATION

Once you create a class type, you can declare one or more objects of that class type. For example:

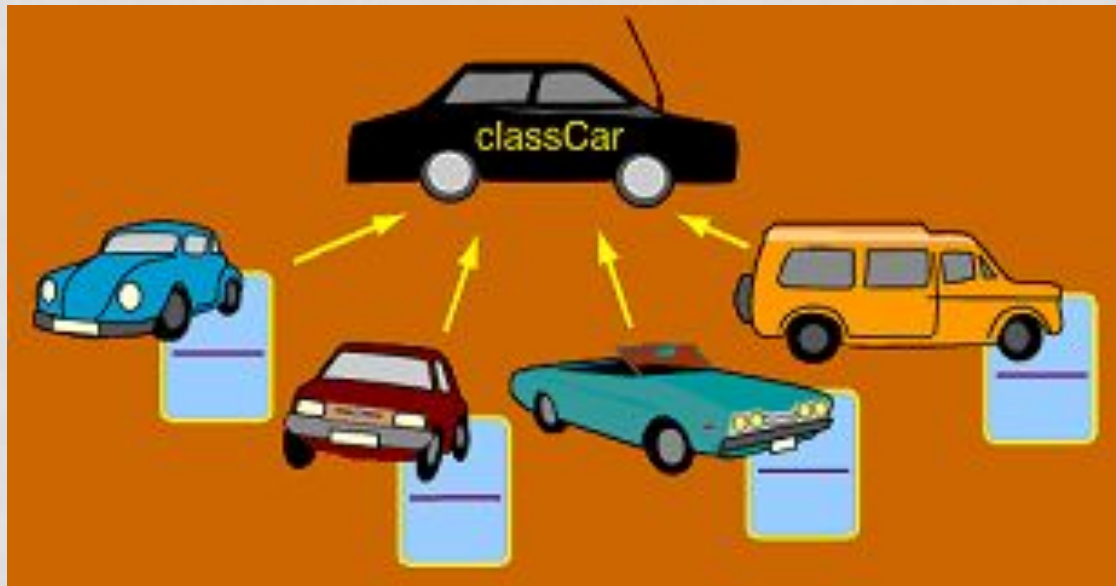
```
class circle{  
    // class body  
}  
Void main()  
{  
    circle c;  
}
```

OBJECTS ?

- An object is an instance of a class.



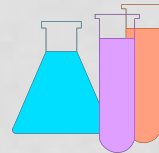
CONTD.



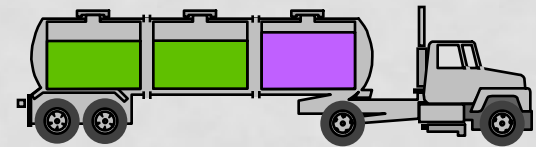
CONTD.

- Informally, an object represents an entity, either physical, conceptual, or software.

- Physical entity

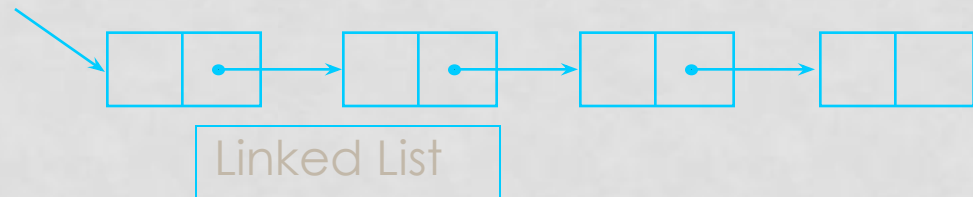


Chemical
Process



Truck

- Conceptual entity



- Software entity

class circle

```
{  
private:  
float radius;  
public:  
circle(){radius=5.5;}  
circle(float r) {set(r);}  
void set(float r)  
{  
    if(r>0)  
    {radius = r;}  
    else  
    {radius=5.5;}  
}
```

```
float calcArea()  
{  
    return 3.14*power(radius,2);  
}  
float getRadius()  
{  
    Return radius;  
} };  
void main()  
{  
    circle c1; //create circles  
    circle c2(6);circle c3(-3);  
    c1.calcArea();  
    cout<<c3.calcArea()<<endl;  
    cout<<c3.getRadius();  
}
```

DEFAULT PARAMETERIZED CONSTRUCTOR

```
Class Circle  
{  
Float rad;  
Circle(float r=5.5)  
{  
rad=r;  
}  
};
```


class circle

```
{  
private:  
float radius;  
public:  
circle(float r=5.5)  
{  
Set(r);  
}  
void set(float r)  
{  
If(r>0)  
{radius = r;}  
else  
{radius=5.5;}  
}
```

```
float calcArea()  
{  
return 3.14*power(radius,2);  
}  
float getRadius()  
{  
Return radius;  
}};  
void main()  
{  
circle c1; //create circles  
circle c2(6); circle c3(-3);  
c1.calcArea();  
cout<<c3.calcArea()<<endl;  
cout<<c3.getRadius();  
}
```

CALL FOR CONSTRUCTOR AND DESTRUCTOR

- When a variable of programmer defined data type is created and constructor is called it will result in an automatic call to constructor of all the variables of programmer defined data type present inside i.e. when a circle type variable is created constructor is called for class circle.
- Similarly whenever scope of variable ends, destructor of the class is called.

CLASS TASK

- Make a class “**Cube**”. Choose appropriate **data members**. Your program should be able to calculate:
 - **Surface Area of cube (formula area= $6a^2$)**
 - **Volume of cube (volume = a^3)**

FAQS

- MS Teams
Queries

- LMS
Lab Task submission
Assignment submission
Project submission

REFERENCE

- Chapter 16 , C How to Program by Deitel & Deitel, 9th Ed.